Software Project Management

Four Common (subjective) estimation models

- Expert Judgment
- Analogy
- Parkinson’s law
- Price to Win

Expert Judgement (Category: Empirical)

- One /more experts in both software development and the application domain use their experience to predict/guess software costs.
- Experts divide a software product into component units. Add up the costs for each of the components.
- Process iterates until some consensus is reached.
- Advantages:
  - Relatively cheap estimation method.
  - Can be accurate if experts have direct experience of similar systems.
- Disadvantages:
  - Very inaccurate if there are no experts!
  - Suffers from individual bias.

Delphi Estimation (Category: Empirical)

- Overcomes some of the problems of expert judgement.
- Team of Experts and a coordinator.
- Experts carry out estimation independently.
  - mention the rationale behind their estimation.
- Coordinator notes down any extraordinary rationale.
  - Circulates among experts.
  - Experts re-estimate.
  - Experts never meet each other to discuss their viewpoints.

Estimation by Analogy

The cost of project is computed by **comparing the project to a similar project** in the same application domain.

- Advantages:
  - May be accurate if project data available and people /tools are the same
- Disadvantages:
  - Impossible if no comparable project has been tackled.
  - Needs systematically maintained cost database

Parkinson's Law

- Project costs whatever resources are available
- Advantages: No overspend.
- Disadvantages: System is usually unfinished.

Cost Pricing to win

- The project costs whatever the customer has to spend on it
- Advantages:
  - You get the contract
- Disadvantages:
  - The probability that the customer gets the system he or she wants is small.
  - Costs do not accurately reflect the work required.
- Difficulty:
  - How do you know what customer has?
- When:
  - Only a good strategy if you are willing to take a serious loss to get a first customer, or
  - if Delivery of a radically reduced product is a real option.

What can we say about Estimation methods?

- Each method has strengths and weaknesses.
- Estimation should be based on several methods.
- If these do not return approximately the same result, then
  - you have insufficient information available to make an estimate.
- Some action should be taken to find out more in order to make more accurate estimates.
- Pricing to win is sometimes the only applicable method.

Pricing to Win

- This approach may seem unethical and un-businesslike.
- However, when detailed information is lacking it may be the only appropriate strategy.
- The project cost is agreed on the basis of an outline proposal and the development is constrained by that cost.
- A detailed specification may be negotiated or an evolutionary approach used for system development.

Top-down and bottom-up estimation

- Either top-down or bottom-up approaches may be used.
- Top-down
  - Start at the system level.
  - Assess the overall system functionality and how this is delivered through sub-systems.
- Bottom-up
  - Start at the component level.
  - Estimate the effort required for each component.
  - Add these efforts to reach a final estimate.

Top-down estimation

- Usable without knowledge of the system architecture and the components that might be part of the system.
- Takes into account costs such as integration, configuration management and documentation.
- Can underestimate the cost of solving difficult low-level technical problems.

Bottom-up estimation

- Usable when the architecture of the system is known and components identified.
- This can be an accurate method if the system has been designed in detail.
- It may underestimate the costs of system level activities such as integration and documentation.

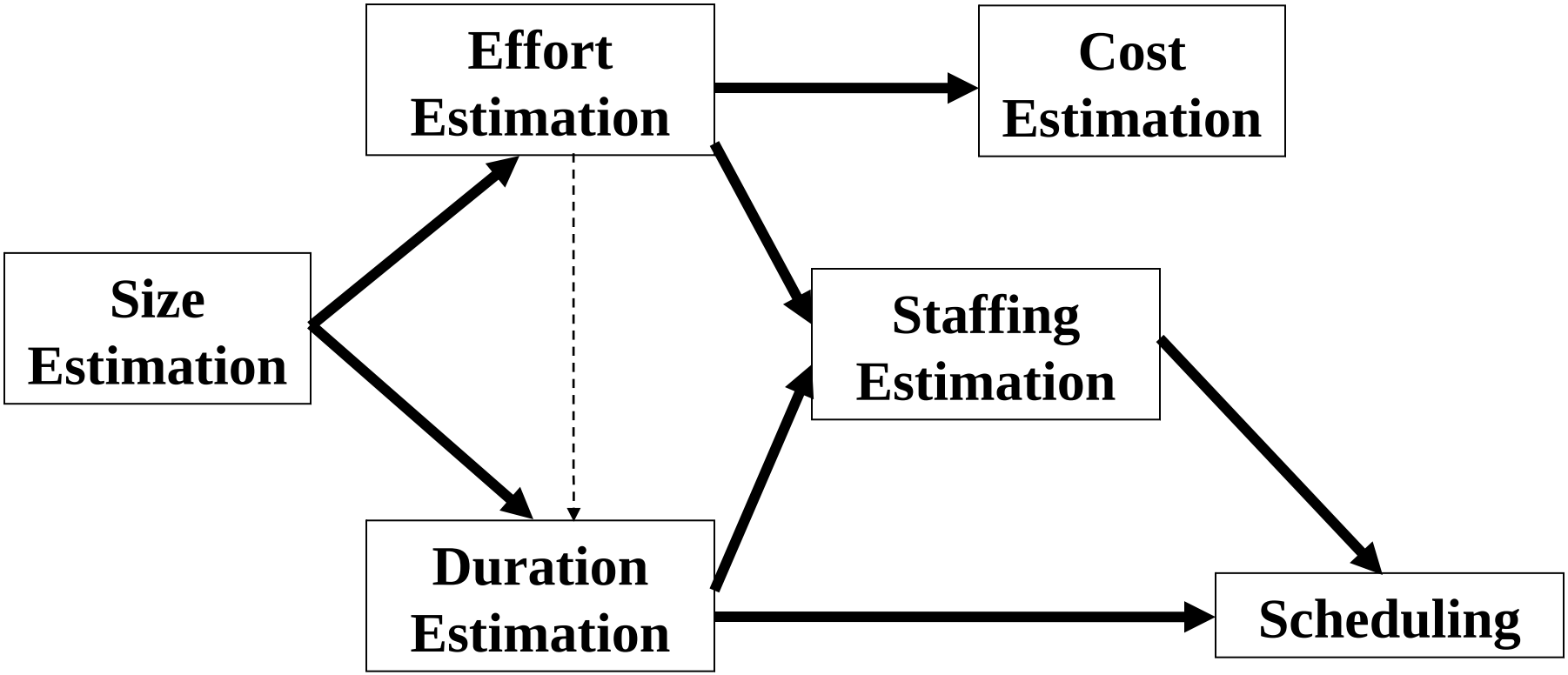
Criteria for a Good Estimation Model

- Defined—clear what is estimated
- Accurate
- Objective—avoids subjective factors
- Results understandable
- Detailed
- Stable
- Right Scope
- Easy to Use
- Causal—future data not required

Software Cost Estimation

- Determine size of the product.
- From the size estimate,
  - determine the effort needed.
- From the effort estimate,
  - determine project duration, and cost.

Software Cost Estimation



Software productivity

- A measure of the rate at which individual engineers involved in software development **produce software and associated documentation**.
- Not quality-oriented although quality assurance is a factor in productivity assessment.
- Essentially, we want to measure useful functionality produced per time unit.

Productivity measures

- Size related measures based on some output from the software process. This may be lines of delivered source code, object code instructions, etc.
- Function-related measures based on an estimate of the functionality of the delivered software. Function-points are the best known of this type of measure.

Factors affecting Productivity

Application domain experience	Knowledge of the application domain is essential for effective software development. Engineers who already understand a domain are likely to be the most productive.
Process quality	The development process used can have a significant effect on productivity.
Project size	The larger a project, the more time required for team communications. Less time is available for development so individual productivity is reduced.
Technology support	Good support technology such as CASE tools, configuration management systems, etc. can improve productivity.
Working environment	A quiet working environment with private work areas contributes to improved productivity.

Quality and Productivity

- All metrics based on volume/unit time are flawed because they do not take quality into account.
- Productivity may generally be increased at the cost of quality.
- It is not clear how productivity/quality metrics are related.
- If requirements are constantly changing then an approach based on counting lines of code is not meaningful as the program itself is not static.

Measurement problems

- Estimating the size of measure (e.g. how many function points).
- Estimating the total number of programmer months that have elapsed.
- Estimating productivity (e.g. documentation team) and incorporating this estimate in overall estimate.

Software Size Metrics - LOC

- This model assumes that there is a linear relationship between system size and volume of documentation.
- What programs should be counted as part of the system?
- Simplest and most widely used metric.
- Comments and blank lines should not be counted.
- How does this correspond to statements as in Java which can span several lines or where there can be several statements on one line.
- Several problems with using LOC as a unit of measure for software.

Disadvantages of using LOC

- Size can vary with coding style.
- Focuses on coding activity alone.
- Correlates poorly with quality and efficiency of code.
- Penalizes higher level programming languages, code reuse, etc.
- Does not address issues of structural or logical complexity.
- Difficult to estimate LOC from problem description - So not useful for early project planning.